

Trame de rapport d'audit — eRezo

Modèle à remplir par votre groupe. Remplacez chaque bloc en italique par votre analyse. Le rapport final est rendu en PDF le **vendredi 17 juillet 2026**, à partir du travail réalisé **en séance**. Soyez concis et factuel : un bon audit tient en 6 à 10 pages.

Page de garde

- **Projet audité** : eRezo — SaaS de gestion de contacts et d'événements
- **Équipe (groupe)** : *noms des membres*
- **Date de l'audit** : *date*
- **Périmètre** : code source (lecture) + production <https://erezo.fr> (lecture seule)
- **Autorisation de test signée** : *oui / n° annexe*

1. Résumé exécutif

En 5-8 lignes, pour un lecteur non technique (le client) :

- *État général de l'application (sain / à surveiller / à risque)*
- *Les 2-3 points les plus importants*
- *Verdict de reprenabilité (reprenable en < 30 min ? oui/non/sous conditions)*

Score de risque global (sécurité) : $X / 10$

Sévérité	Nombre de failles
Critique	<i>n</i>
Haute	<i>n</i>
Moyenne	<i>n</i>
Basse / Info	<i>n</i>

2. Architecture & conception

Domaines métier identifiés (bounded contexts) : Listez les grands domaines (ex : gestion de contacts, événements, messagerie, comptes...) et dites s'ils sont bien délimités.

Style d'architecture observé : Monolithe ? Modulaire ? Trace d'architecture hexagonale (séparation domaine / infrastructure, ports & adapters) ? Justifiez avec des exemples de dossiers/fichiers.

Ubiquitous language : Le code emploie-t-il les termes du métier ? Exemples.

Points de couplage / dette technique : Où est-ce rigide ? Qu'est-ce qui serait risqué à faire évoluer ?

Verdict architecture : / points forts / points faibles / recommandations

3. Tests & qualité

Tests présents : Types (unitaires / intégration / E2E), où ils se trouvent, ce qu'ils couvrent.

Pyramide des tests : Respectée ? Inversée ? Absente ?

Traces de TDD : Voit-on des tests écrits avant le code ? Le cœur métier est-il testé ?

Outils qualité : CI, linters, conventions, hooks...

Verdict tests : / recommandations

4. Stack technique

Décrivez la stack que vous avez identifiée :

Couche	Technologie identifiée	Comment vous l'avez découverte
Langage / runtime	ex : ...	headers HTTP, code, ...
Framework backend	...	...
Base de données	...	...
Frontend / templating	...	...
Hébergement / infra / CDN	...	...
Services externes	...	...

Analyse des choix : cohérents avec un SaaS de ce type ? Ce que vous feriez autrement.

5. Audit de sécurité (pentest IA — OWASP Top 10)

Rappel : **lecture seule, non destructif**. Aucune donnée personnelle réelle ne doit apparaître dans ce rapport (anonymisez).

Méthodologie : outils / agents IA utilisés, phases (reco → tests → vérification).

Findings

Pour **chaque faille** trouvée, remplissez :

[FIND-00X] Titre

- **Sévérité** : Critique / Haute / Moyenne / Basse · **Priorité** : P0 / P1 / P2
- **Endpoint / zone concernée** : ...
- **Description** : ...
- **Preuve** : requête, capture anonymisée, comportement observé
- **Impact** : ce que ça permet à un attaquant
- **Correction recommandée** : ...

(Dupliquez ce bloc pour chaque finding.)

Points positifs (ce qui est bien sécurisé)

Listez ce qui résiste (ex : CSRF, cookies, échappement XSS...). Un bon auditeur note aussi ce qui va bien.

Plan de remédiation priorisé

Priorité	Finding	Effort estimé	Impact
P0	...	faible/moyen/élevé	...
P1	...	...	...
P2	...	...	...

6. Reprenabilité (« test du freelance »)

- **Documentation présente** : *CLAUDE.md* ? *README* ? *doc d'onboarding* ?
 - **Temps estimé pour comprendre + lancer le projet** : *< 30 min* ? *plus* ?
 - **Ce qui manque pour rendre le projet reprenable** : ...
 - **Verdict** : *reprenable* / *partiellement* / *non* — *argumenté*
-

7. Synthèse des recommandations

Top 5 des actions prioritaires, tous axes confondus, dans l'ordre où vous conseillez au client de les traiter.

1. ...
 2. ...
 3. ...
 4. ...
 5. ...
-

Annexe — Audit croisé

Commentaires reçus d'un autre groupe sur votre rapport (séance 5) et comment vous les intégrez.